PYTHON FOR AI · FOUNDATIONS

파이썬 기초부터 이미지처리까지

Python · 클래스 · NumPy · Pandas · 이미지처리 이론

AI 개발을 시작하는 데 필요한 파이썬 문법, 객체지향 프로그래밍, 데이터 처리, 이미지처리 이론까지 한 번에 다집니다.

CONTENTS

목차

1 **파이썬 기초** 03 – 45

변수·자료형·제어문·함수·고차함수·예외처리까지 파이썬 문법의 기본기를 다집니다

2 **클래스와 객체지향** 46 – 78

클래스·상속·다형성과 AI 코드에서 클래스가 쓰이는 방식을 익힙니다

3 **NumPy** 79 – 99

다차원 배열과 벡터·행렬 연산으로 AI 연산의 기초를 다룹니다

4 **Pandas** 100 – 120

표 형태 데이터를 불러오고 탐색·가공·집계하는 방법을 익힙니다

5 **이미지처리 이론** 121 – 138

픽셀·색상공간부터 컨볼루션·정규화까지 비전 AI의 기초를 다룹니다

파이썬 · 클래스 · NumPy · Pandas · 이미지처리 기초

PART

1

파이썬 기초

AI 개발의 출발점, 파이썬 문법부터 자료구조, 제어문, 함수, 예외처리까지 기초를 탄탄히 다집니다.

파이썬이란?

파이썬은 문법이 간결하고 배우기 쉬운 인터프리터 언어로, 데이터 분석과 AI 개발 분야에서 사실상의 표준 언어로 자리잡았습니다.

인터프리터 언어

코드를 한 줄씩 바로 실행할 수 있어 학습과 실험에 유리합니다.

AI·데이터 분야 표준

NumPy, Pandas, PyTorch 등 주요 AI 라이브러리가 모두 파이썬 기반입니다.

간결한 문법

세미콜론이나 중괄호 없이도 읽기 쉬운 코드를 작성할 수 있습니다.

방대한 생태계

PyPI에 수십만 개의 패키지가 등록되어 있어 거의 모든 기능을 가져다 쓸 수 있습니다.

개발 환경 확인하기

확인

bash (.venv)

```
$ source .venv/bin/activate  이전 가이드에서 만든 가상환경 활성화
$ python --version  파이썬 버전 확인
$ python  대화형 인터프리터 실행
```

NOTE 우분투 개발환경 구축 가이드에서 만든 .venv를 그대로 사용합니다. 별도 환경이 없다면 먼저 그 가이드를 참고하세요.

파이썬 기본 자료형

자료형	설명	예시
int	정수(소수점 없는 숫자)	age = 25
float	실수(소수점이 있는 숫자)	pi = 3.14
str	문자열(따옴표로 감싼 텍스트)	name = 'AI'
bool	참/거짓 논리값	is_ready = True
NoneType	값이 없음을 나타냄	result = None

변수 선언과 대입

코드

variables.py

```
name = "지우"
age = 28
height = 168.5

# 여러 변수를 한 줄에 대입
x, y, z = 1, 2, 3

print(name, age, height)
```

문자열 인덱싱과 슬라이싱

코드

strings.py

```
s = "Python AI"
print(s[0])          # P
print(s[-1])         # I
print(s[0:6])        # Python
print(s[7:])         # AI
print(s.upper())     # PYTHON AI
print(f"안녕, {s}!")
```

자주 쓰는 문자열 메서드

코드

항목	설명
`.strip()`	문자열 양 끝의 공백을 제거합니다.
`.split(",")`	구분자를 기준으로 문자열을 리스트로 나눕니다.
`"-".join(list)`	리스트의 요소들을 구분자로 이어 붙입니다.
`.replace(a, b)`	문자열 안의 a를 b로 모두 바꿉니다.
`.format() / f-string`	변수 값을 문자열 안에 끼워 넣습니다.

연산자 한눈에 보기

코드

항목	설명
`+ - * / // % **`	산술 연산자 (몫 //, 나머지 %, 거듭제곱 **)
`== != > < >= <=`	비교 연산자 — 결과는 항상 bool
`and or not`	논리 연산자 — 조건을 조합할 때 사용
`= += -= *= /=`	대입 연산자 — 변수에 값을 갱신하며 대입
`in / not in`	멤버십 연산자 — 리스트·문자열 안에 있는지 확인

자료형 변환하기

코드

casting.py

```
age_str = "28"
age_int = int(age_str)       # "28" -> 28
price = float("19900")      # "19900" -> 19900.0
text = str(3.14)            # 3.14 -> "3.14"

print(age_int + 2)           # 30
print(type(age_int))         # <class 'int'>
```

직접 해보기

실습

TASK

변수 name, age, height를 선언하고, f-string으로 "OO님은 XX세이고 키는 YYcm입니다"를 출력해보세요.

TIP

문자열과 숫자를 함께 출력할 때는 f"{변수}" 형태를 쓰면 형변환 없이 바로 넣을 수 있습니다.

리스트 vs 튜플

개념

리스트 (list)

- [ ] 로 만들며 순서가 있는 자료구조입니다.
- 값을 추가·수정·삭제할 수 있습니다 (mutable).
- 장바구니처럼 계속 바뀌는 데이터에 적합합니다.

튜플 (tuple)

- ( ) 로 만들며 순서가 있는 자료구조입니다.
- 한 번 만들면 값을 바꿀 수 없습니다 (immutable).
- 좌표처럼 변하지 않는 값 묶음에 적합합니다.

리스트 다루기

코드

lists.py

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")      # 끝에 추가
fruits.insert(1, "grape")   # 1번 위치에 삽입
fruits.pop()                # 마지막 요소 꺼내기

print(fruits[0:2])          # 슬라이싱
print(len(fruits))          # 리스트 길이
```

자주 쓰는 리스트 메서드

코드

항목	설명
`.append(x)`	리스트 끝에 요소 x를 추가합니다.
`.remove(x)`	값이 x인 첫 번째 요소를 제거합니다.
`.sort()`	리스트를 오름차순으로 정렬합니다.
`.reverse()`	리스트의 순서를 뒤집습니다.
`.index(x)`	값 x가 있는 위치(인덱스)를 반환합니다.

튜플과 집합(set)

코드

tuple_set.py

```
point = (3, 5)                # 튜플: 변하지 않는 값
x, y = point                  # 튜플 언패킹

tags = {"ai", "python", "ai"}  # 집합: 중복 자동 제거
print(tags)                   # {'ai', 'python'}
tags.add("numpy")
```

딕셔너리 다루기

코드

dicts.py

```
student = {"name": "지우", "age": 24, "major": "AI"}

print(student["name"])        # 키로 값 조회
student["age"] = 25           # 값 수정
student["gpa"] = 4.0          # 새 키 추가

for key, value in student.items():
    print(key, ":", value)
```

자주 쓰는 딕셔너리 메서드

코드

항목	설명
`.keys()`	딕셔너리의 모든 키를 반환합니다.
`.values()`	딕셔너리의 모든 값을 반환합니다.
`.items()`	(키, 값) 쌍을 함께 반환합니다.
`.get(key, 기본값)`	키가 없어도 오류 없이 기본값을 반환합니다.
`.update({...})`	다른 딕셔너리의 내용으로 갱신합니다.

직접 해보기

실습

TASK

학생 3명의 이름과 점수를 담은 딕셔너리를 리스트에 담고, for문으로 각 학생의 이름과 점수를 출력해보세요.

TIP

딕셔너리를 담은 리스트는 [{"name":"A","score":90}, ...] 형태로, 실무 데이터에서 아주 자주 등장하는 구조입니다.

if / elif / else 조건문

코드

conditions.py

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"

print(f"{score}점 -> {grade}학점")
```

for 반복문

코드

for_loop.py

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)

for i in range(5):          # 0부터 4까지
    print(i, "번째 반복")
```

while 반복문과 break·continue

코드

while_loop.py

```
count = 0
while count < 5:
    if count == 3:
        count += 1
        continue          # 3만 건너뛰기
    print(count)
    count += 1

# break는 반복문을 즉시 종료합니다
```

range · enumerate · zip

코드

loop_tools.py

```
names = ["지우", "민준", "서연"]
scores = [90, 85, 95]

for i, name in enumerate(names):      # 인덱스와 값
    print(i, name)

for name, score in zip(names, scores):  # 두 리스트 동시에
    print(name, score)
```

리스트 컴프리헨션

코드

comprehension.py

```
nums = [1, 2, 3, 4, 5]

squares = [n**2 for n in nums]
print(squares)                # [1, 4, 9, 16, 25]

evens = [n for n in nums if n % 2 == 0]
print(evens)                  # [2, 4]
```

직접 해보기

실습

TASK

1부터 20까지의 숫자 중 3의 배수만 리스트 컴프리헨션으로 뽑아 출력해보세요.

TIP

조건은 [식 for 변수 in 범위 if 조건] 순서로 씁니다. range(1, 21)로 1~20 범위를 만들 수 있습니다.

함수 정의 기초

코드

functions.py

```
def greet(name):
    return f"안녕하세요, {name}님!"

message = greet("지우")
print(message)

def add(a, b):                  # 여러 값을 계산해 반환
    return a + b
```

매개변수 활용하기

코드

params.py

```
def power(base, exp=2):        # 기본값 있는 매개변수
    return base ** exp

print(power(3))                # exp 생략 -> 9
print(power(3, 3))             # 27

def total(*args):              # 개수 제한 없는 인자
    return sum(args)

print(total(1, 2, 3, 4))       # 10
```

지역변수 vs 전역변수

개념

지역변수 (local)

- 함수 안에서 선언된 변수입니다.
- 해당 함수 안에서만 사용할 수 있습니다.
- 함수가 끝나면 사라집니다.

전역변수 (global)

- 함수 밖(모듈 최상단)에서 선언된 변수입니다.
- 프로그램 전체에서 접근할 수 있습니다.
- 함수 안에서 수정하려면 global 키워드가 필요합니다.

람다 함수

코드

lambda.py

```
square = lambda x: x ** 2
print(square(5))              # 25

nums = [5, 2, 8, 1]
nums.sort(key=lambda x: -x)   # 내림차순 정렬
print(nums)                   # [8, 5, 2, 1]
```

여러 값 반환하기

코드

multi_return.py

```
def get_stats(nums):
    return min(nums), max(nums), sum(nums) / len(nums)

lo, hi, avg = get_stats([3, 7, 2, 9])
print(lo, hi, avg)          # 2 9 5.25
```

NOTE 콤마로 여러 값을 반환하면 자동으로 튜플로 묶이고, 받는 쪽에서 여러 변수로 바로 나눠 받을 수 있습니다.

재귀 함수

코드

recursion.py

```
def factorial(n):
    if n <= 1:
        return 1          # 멈추는 조건(base case)
    return n * factorial(n - 1)

print(factorial(5))       # 120
```

NOTE 재귀 함수는 자기 자신을 다시 호출합니다. 반드시 멈추는 조건이 있어야 무한 반복에 빠지지 않습니다.

map()으로 일괄 적용하기

코드

map_ex.py

```
nums = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x ** 2, nums))
print(squares)                # [1, 4, 9, 16, 25]

names = ["ai", "python", "numpy"]
upper = list(map(str.upper, names))
print(upper)                  # ['AI', 'PYTHON', 'NUMPY']
```

NOTE map(함수, 리스트)는 리스트의 모든 요소에 함수를 하나씩 적용해 새로운 결과를 만듭니다.

filter()로 조건에 맞는 값만 걸러내기

코드

filter_ex.py

```
nums = [12, 7, 33, 4, 19, 26]

evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)                    # [12, 4, 26]

def is_adult(age):
    return age >= 20

adults = list(filter(is_adult, [15, 22, 18, 30]))
print(adults)                   # [22, 30]
```

NOTE filter(조건함수, 리스트)는 조건 함수가 True를 반환하는 요소만 남깁니다.

zip()으로 여러 리스트 묶기

코드

zip_ex.py

```
names = ["지우", "민준", "서연"]
scores = [90, 85, 95]

paired = list(zip(names, scores))
print(paired)       # [('지우',90), ('민준',85), ('서연',95)]

score_dict = dict(zip(names, scores))
print(score_dict)   # {'지우':90, '민준':85, '서연':95}
```

NOTE zip은 길이가 다른 리스트를 묶으면 더 짧은 쪽 길이에 맞춰서 잘라냅니다.

map · filter · zip 함께 쓰기

코드

combo_ex.py

```
prices = [1000, 2000, 3000]
discounts = [0.1, 0.2, 0.0]

final_prices = list(map(lambda p, d: p * (1 - d), prices, discounts))
print(final_prices)        # [900.0, 1600.0, 3000.0]

cheap = list(filter(lambda p: p < 2000, final_prices))
print(cheap)               # [900.0, 1600.0]
```

NOTE map()에 리스트를 두 개 이상 넣으면, zip처럼 같은 위치의 값끼리 짝지어 함수에 전달합니다.

functools.reduce로 누적 계산하기

코드

reduce_ex.py

```
from functools import reduce

nums = [1, 2, 3, 4, 5]

total = reduce(lambda acc, x: acc + x, nums)
print(total)                  # 15

maximum = reduce(lambda acc, x: acc if acc > x else x, nums)
print(maximum)                # 5
```

NOTE reduce는 리스트를 왼쪽부터 차례로 누적시켜 하나의 값으로 줄입니다. 합계·최댓값처럼 "하나로 모으는" 계산에 적합합니다.

일반 함수 vs 람다 함수

개념

일반 함수 (def)

- 이름을 붙여 여러 줄로 정의할 수 있습니다.
- 복잡한 로직, 재사용이 잦은 기능에 적합합니다.
- docstring으로 설명을 남기기 좋습니다.

람다 함수 (lambda)

- 이름 없이 한 줄로 간단히 정의합니다.
- map·filter·sort의 key처럼 잠깐 쓰고 버릴 함수에 적합합니다.
- 조건이 복잡해지면 오히려 일반 함수가 더 읽기 쉽습니다.

함수를 인자로 전달하기

코드

higher_order.py

```
def apply_twice(func, x):
    return func(func(x))

print(apply_twice(lambda x: x + 3, 10))     # 16

def run_pipeline(data, steps):
    for step in steps:
        data = step(data)
    return data

result = run_pipeline([1, 2, 3], [lambda d: [x*2 for x in d], sum])
print(result)                                # 12
```

NOTE 함수를 다른 함수의 인자로 넘기는 방식은 데이터 전처리 파이프라인을 구성할 때 자주 쓰입니다.

직접 해보기

실습

TASK

학생 점수 리스트에서 filter로 80점 이상인 점수만 뽑고, map으로 각 점수에 5점을 보너스로 더한 뒤, zip으로 학생 이름과 짝지어 출력해보세요.

TIP

filter와 map은 결과가 바로 보이지 않는 지연 실행(lazy) 객체이므로, list()로 감싸야 눈으로 확인할 수 있습니다.

모듈 불러오기

코드

modules.py

```
import math
from datetime import datetime
import numpy as np            # 관례적으로 별칭을 붙입니다

print(math.sqrt(16))          # 4.0
print(datetime.now())         # 현재 날짜와 시간
```

자주 쓰는 표준 라이브러리

코드

항목	설명
`os`	파일·폴더 경로, 환경변수 등 운영체제 기능을 다룹니다.
`sys`	인터프리터 관련 정보와 실행 인자를 다룹니다.
`math`	제곱근, 삼각함수 등 수학 함수를 제공합니다.
`random`	난수 생성, 무작위 선택 기능을 제공합니다.
`datetime`	날짜와 시간을 다룹니다.
`json`	JSON 형식 데이터를 읽고 씁니다.

try / except로 예외 처리하기

코드

exceptions.py

```
try:
    num = int(input("숫자 입력: "))
    result = 10 / num
except ValueError:
    print("숫자가 아닙니다.")
except ZeroDivisionError:
    print("0으로 나눌 수 없습니다.")
finally:
    print("입력 처리 완료")
```

자주 만나는 예외 유형

코드

항목	설명
`TypeError`	자료형이 맞지 않는 연산을 할 때 발생합니다.
`ValueError`	값의 형식이 올바르지 않을 때 발생합니다.
`IndexError`	리스트 범위를 벗어난 인덱스에 접근할 때 발생합니다.
`KeyError`	딕셔너리에 없는 키로 접근할 때 발생합니다.
`ZeroDivisionError`	숫자를 0으로 나눌 때 발생합니다.

파일 읽고 쓰기

코드

file_io.py

```
with open("notes.txt", "w") as f:
    f.write("AI 개발 공부 중\n")

with open("notes.txt", "r") as f:
    content = f.read()
    print(content)

# with 문을 쓰면 파일을 자동으로 닫아줍니다
```

직접 해보기

실습

TASK

학생 이름과 점수를 파일에서 한 줄씩 읽어(name,score 형식) 딕셔너리 리스트로 만들고, 예외 처리와 함께 평균 점수를 계산해 출력해보세요.

TIP

파일이 없을 수도 있으니 try/except FileNotFoundError로 감싸는 습관을 들이면 실전 코드에서 안전합니다.

PART

2

클래스와 객체지향

데이터와 동작을 하나로 묶는 객체지향 프로그래밍 개념을 익히고, AI 코드에서 클래스가 어떻게 쓰이는지 살펴봅니다.

객체지향 프로그래밍이란?

객체지향(OOP)은 관련된 데이터와 동작(함수)을 하나의 "객체"로 묶어서 다루는 프로그래밍 방식입니다. 대부분의 AI 프레임워크가 이 방식으로 설계되어 있습니다.

데이터+동작을 하나로

속성(데이터)과 메서드(동작)를 한 곳에 모아 관리합니다.

재사용성

한 번 만든 클래스를 여러 곳에서 반복해서 사용할 수 있습니다.

유지보수 용이

관련 코드가 한 곳에 모여 있어 수정과 확장이 쉽습니다.

AI 프레임워크의 기본

PyTorch의 nn.Module, scikit-learn의 모델 클래스가 모두 이 구조입니다.

클래스 vs 인스턴스

개념

클래스 (class)

- 객체를 만들기 위한 '설계도'입니다.
- 붕어빵 틀에 비유할 수 있습니다.
- 속성과 메서드의 틀을 정의합니다.

인스턴스 (instance)

- 클래스로부터 실제로 만들어진 '객체'입니다.
- 틀로 찍어낸 붕어빵 하나하나에 비유됩니다.
- 인스턴스마다 독립적인 데이터를 가집니다.

첫 클래스 정의하기

코드

first_class.py

```
class Dog:
    def __init__(self, name, age):
        self.name = name          # 속성(데이터)
        self.age = age

my_dog = Dog("초코", 3)           # 인스턴스 생성
print(my_dog.name, my_dog.age)   # 초코 3
```

속성과 메서드

코드

methods.py

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):                # 메서드(동작)
        return f"{self.name}: 멍멍!"

my_dog = Dog("초코", 3)
print(my_dog.bark())               # 초코: 멍멍!
```

직접 해보기

실습

TASK

Person 클래스를 만들어 name, age 속성과, 자기소개 문장을 반환하는 introduce() 메서드를 작성해보세요.

TIP

메서드의 첫 번째 매개변수는 항상 self입니다. self를 빠뜨리면 오류가 나니 꼭 챙겨주세요.

클래스 변수 vs 인스턴스 변수

개념

클래스 변수

- 클래스 전체가 공유하는 변수입니다.
- 모든 인스턴스가 같은 값을 참조합니다.
- class 블록 바로 아래에 선언합니다.

인스턴스 변수

- 각 인스턴스마다 독립적인 변수입니다.
- 보통 __init__ 안에서 self.으로 선언합니다.
- 인스턴스마다 서로 다른 값을 가질 수 있습니다.

클래스 변수 예제

코드

class_var.py

```
class Dog:
    species = "Canis familiaris"   # 클래스 변수

    def __init__(self, name):
        self.name = name           # 인스턴스 변수

a = Dog("초코")
b = Dog("보리")
print(a.species, b.species)        # 둘 다 동일
```

자주 쓰는 매직 메서드

코드

항목	설명
`__init__`	인스턴스가 생성될 때 자동으로 호출되는 생성자입니다.
`__str__`	print()로 출력할 때 보여줄 문자열을 정의합니다.
`__repr__`	개발자가 보는 객체의 공식적인 표현을 정의합니다.
`__len__`	len()을 호출했을 때의 동작을 정의합니다.
`__eq__`	== 연산자로 비교할 때의 동작을 정의합니다.

__str__로 출력 형식 정하기

코드

magic_str.py

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"Dog(이름={self.name}, 나이={self.age})"

print(Dog("초코", 3))    # Dog(이름=초코, 나이=3)
```

캡슐화란

개념

공개 속성 (public)

- 밑줄 없이 선언하는 일반적인 속성입니다.
- 클래스 밖에서 자유롭게 접근·수정할 수 있습니다.
- 예: self.name

비공개 속성 (private)

- 이름 앞에 밑줄을 붙여 직접 건드리지 말라고 표시합니다.
- _name은 관례적 비공개, __name은 강하게 숨김 처리됩니다.
- 안전한 값 검증을 위해 property와 함께 자주 씁니다.

property로 값 검증하기

코드

property_ex.py

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("잔액은 음수일 수 없습니다")
        self._balance = value
```

NOTE @property와 @속성명.setter를 함께 쓰면 값을 읽고 쓸 때마다 검증 로직을 끼워넣을 수 있습니다.

상속이란

개념

부모 클래스 (Base)

- 공통 속성과 메서드를 가진 클래스입니다.
- 여러 자식 클래스가 이를 물려받습니다.
- 예: Animal 클래스

자식 클래스 (Derived)

- 부모 클래스를 물려받아 기능을 확장합니다.
- 부모의 속성·메서드를 그대로 쓰거나 재정의할 수 있습니다.
- 예: Dog, Cat 클래스가 Animal을 상속

상속 기본 문법

코드

inheritance.py

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "..."

class Dog(Animal):                # Animal을 상속
    def speak(self):
        return f"{self.name}: 멍멍!"

print(Dog("초코").speak())
```

super()로 부모 기능 활용하기

코드

super_ex.py

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)      # 부모 생성자 호출
        self.breed = breed

d = Dog("초코", "푸들")
print(d.name, d.breed)
```

메서드 오버라이딩

코드

override.py

```
class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):                # 부모 메서드 재정의
        return 3.14 * self.r ** 2

print(Circle(3).area())            # 28.26
```

직접 해보기

실습

TASK

Shape을 상속받는 Rectangle 클래스를 만들어 width, height를 받고 area() 메서드로 넓이를 반환해보세요.

TIP

부모 클래스의 __init__을 그대로 쓸지, super()로 확장할지는 상황에 따라 다릅니다. 필요한 속성이 늘어나면 super()를 활용하세요.

다형성이란

개념

같은 이름, 다른 동작

- 서로 다른 클래스가 같은 이름의 메서드를 가질 수 있습니다.
- 호출하는 쪽은 어떤 클래스인지 몰라도 동일하게 호출합니다.
- 예: 모든 도형이 area()를 갖지만 계산 방식은 다릅니다.

왜 유용한가

- 도형·모델 종류가 늘어나도 호출 코드는 바뀌지 않습니다.
- for shape in shapes: shape.area() 처럼 통일된 코드 작성이 가능합니다.
- AI에서도 다양한 모델 클래스를 동일한 인터페이스로 다룹니다.

추상 클래스로 규칙 강제하기

코드

abstract.py

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass            # 자식이 반드시 구현

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2
```

NOTE Shape 자체는 인스턴스로 만들 수 없고, area()를 구현하지 않은 자식 클래스도 만들 수 없습니다.

인스턴스·클래스·스태틱 메서드

코드

항목	설명
`def method(self)`	인스턴스 메서드 — 인스턴스 데이터(self)에 접근합니다.
`@classmethod`	클래스 자체(cls)를 다루며, 대안 생성자로도 자주 쓰입니다.
`@staticmethod`	self·cls 없이 클래스와 관련된 독립적인 함수를 정의합니다.

classmethod와 staticmethod

코드

method_types.py

```
class Dog:
    count = 0

    def __init__(self, name):
        self.name = name
        Dog.count += 1

    @classmethod
    def total(cls):
        return f"지금까지 {cls.count}마리"

    @staticmethod
    def is_valid_name(name):
        return len(name) > 0
```

dataclass로 간결하게 만들기

코드

dataclass_ex.py

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

p = Point(3, 5)
print(p)          # Point(x=3, y=5) — __repr__ 자동 생성
```

NOTE @dataclass를 쓰면 __init__, __repr__, __eq__ 등을 직접 작성하지 않아도 자동으로 생성됩니다.

왜 AI 개발에서 클래스를 쓰는가

데이터셋, 전처리, 모델을 각각 클래스로 분리하면 코드가 명확해지고, 다른 프로젝트에서도 그대로 재사용할 수 있습니다.

데이터셋 클래스

원본 데이터를 불러오고 전처리하는 로직을 하나로 묶습니다.

모델 wrapper 클래스

학습·예측 함수를 감싸서 사용하기 쉽게 만듭니다.

PyTorch·scikit-learn

실제로 이 라이브러리들의 핵심 구조가 모두 클래스 기반입니다.

설정값 관리

하이퍼파라미터 같은 설정을 속성으로 묶어 관리하기 편합니다.

실전 예제 — Dataset 클래스

코드

dataset.py

```
class ImageDataset:
    def __init__(self, file_paths, labels):
        self.file_paths = file_paths
        self.labels = labels

    def __len__(self):
        return len(self.file_paths)

    def __getitem__(self, idx):
        return self.file_paths[idx], self.labels[idx]
```

NOTE __len__과 __getitem__을 구현하면 len(dataset), dataset[0]처럼 자연스럽게 사용할 수 있습니다.

실전 예제 — Model wrapper 클래스

코드

model_wrapper.py

```
class SimpleClassifier:
    def __init__(self, model, threshold=0.5):
        self.model = model
        self.threshold = threshold

    def predict(self, x):
        score = self.model(x)
        return "양성" if score > self.threshold else "음성"
```

클래스 조합해서 쓰기

코드

compose.py

```
dataset = ImageDataset(paths, labels)
clf = SimpleClassifier(my_model, threshold=0.6)

for i in range(len(dataset)):
    path, label = dataset[i]
    result = clf.predict(load_image(path))
    print(path, "->", result)
```

NOTE 각 클래스가 자신의 역할만 책임지므로, 데이터나 모델이 바뀌어도 다른 부분을 건드릴 필요가 없습니다.

직접 해보기

실습

TASK

내가 관심 있는 주제(예: 도서, 영화, 운동 기록)로 클래스를 하나 설계해, 속성 3개 이상과 메서드 2개 이상을 작성해보세요.

TIP

처음부터 완벽하게 설계하려 하지 말고, __init__과 __str__부터 만든 뒤 필요한 메서드를 하나씩 추가해보세요.

컴포지션 vs 상속

개념

상속 (is-a)

- '~는 ~이다' 관계일 때 사용합니다. (Dog는 Animal이다)
- 부모의 기능을 그대로 물려받습니다.
- 계층이 깊어지면 오히려 복잡해질 수 있습니다.

컴포지션 (has-a)

- '~는 ~를 가진다' 관계일 때 사용합니다. (Car는 Engine을 가진다)
- 클래스 안에 다른 클래스의 인스턴스를 속성으로 둡니다.
- 더 유연하고 느슨하게 결합된 구조를 만들 수 있습니다.

클래스 작성 시 자주 하는 실수

코드

항목	설명
`self` 빠뜨리기	메서드의 첫 매개변수 self를 빠뜨리면 TypeError가 발생합니다.
가변 기본값	def __init__(self, items=[])처럼 리스트를 기본값으로 쓰면 안 됩니다.
`__init__` 오타	앞뒤 밑줄 두 개(__init__)를 정확히 써야 자동으로 호출됩니다.
변수 범위 혼동	가변 객체를 클래스 변수로 두면 모든 인스턴스가 값을 공유해버립니다.

나만의 예외 클래스 만들기

코드

custom_exception.py

```
class InvalidScoreError(Exception):
    pass

def set_score(score):
    if not (0 <= score <= 100):
        raise InvalidScoreError("점수는 0~100 사이여야 합니다")
    return score

set_score(150)   # InvalidScoreError 발생
```

함수형 vs 객체지향 스타일

개념

함수 중심

- 간단한 스크립트나 일회성 처리에 적합합니다.
- 데이터와 로직이 분리되어 있습니다.
- 상태를 유지할 필요가 없을 때 더 간결합니다.

객체지향 중심

- 상태(데이터)를 계속 유지·관리해야 할 때 유리합니다.
- 비슷한 대상이 여러 개 반복될 때 적합합니다.
- 실전 AI 프로젝트에서는 둘을 함께 섞어 씁니다.

직접 해보기

실습

TASK

Animal 부모 클래스와 Dog, Cat 자식 클래스를 만들어 다형성을 확인해보세요. animals 리스트에 담아 for문으로 각각의 speak()를 호출해보세요.

TIP

부모 타입의 리스트 하나로 여러 자식 클래스를 동일하게 다룰 수 있다는 것이 다형성의 핵심입니다.

객체지향 핵심 개념 총정리

요약

항목	설명
`class / self`	클래스 정의와 인스턴스 자기 자신을 가리키는 키워드
`__init__ / __str__`	생성자와 출력 형식을 정의하는 매직 메서드
상속 / `super()`	부모 클래스의 기능을 물려받고 확장하는 방법
다형성 / 추상클래스	같은 인터페이스로 다양한 객체를 다루는 방법
`classmethod / staticmethod`	인스턴스 없이도 호출 가능한 메서드 종류
`dataclass`	반복적인 클래스 코드를 줄여주는 데코레이터

PART

3

NumPy

AI·데이터 연산의 기본이 되는 다차원 배열과 벡터·행렬 연산을 NumPy로 다뤄 봅니다.

NumPy란?

NumPy는 파이썬에서 다차원 배열을 빠르게 계산할 수 있게 해주는 라이브러리로, 거의 모든 AI·데이터 라이브러리의 기반이 됩니다.

다차원 배열(ndarray)

벡터, 행렬, 텐서 같은 수치 데이터를 효율적으로 표현합니다.

빠른 연산 속도

C로 구현되어 있어 순수 파이썬 반복문보다 훨씬 빠릅니다.

AI 라이브러리의 기반

Pandas, PyTorch, TensorFlow 등이 내부적으로 NumPy 방식을 따릅니다.

벡터·행렬 연산

AI 모델의 계산은 대부분 벡터·행렬 연산으로 이루어집니다.

설치하고 불러오기

확인

bash (.venv)

```
$ pip install numpy  NumPy 설치
$ python -c "import numpy as np; print(np.__version__)"  설치 확인
```

NOTE 관례적으로 import numpy as np로 불러와 np라는 별칭을 사용합니다.

ndarray 만들기

코드

create_array.py

```
import numpy as np

a = np.array([1, 2, 3])              # 리스트로부터 생성
b = np.zeros((2, 3))                  # 0으로 채운 2x3 배열
c = np.ones((3, 3))                   # 1로 채운 3x3 배열
d = np.arange(0, 10, 2)               # 0,2,4,6,8
e = np.linspace(0, 1, 5)              # 0~1을 5등분
```

배열의 주요 속성

코드

항목	설명
`.shape`	각 차원의 크기를 튜플로 반환합니다. 예: (2, 3)
`.ndim`	배열의 차원 수를 반환합니다.
`.dtype`	배열 요소의 자료형을 반환합니다 (예: int64, float32).
`.size`	배열에 담긴 전체 요소의 개수를 반환합니다.

인덱싱과 슬라이싱

코드

indexing.py

```
a = np.array([[1, 2, 3], [4, 5, 6]])

print(a[0, 1])          # 2 — 0행 1열
print(a[:, 1])          # [2 5] — 모든 행의 1열
print(a[1, :])          # [4 5 6] — 1행 전체
print(a[0:2, 1:3])      # 부분 배열 슬라이싱
```

불리언·팬시 인덱싱

코드

fancy_indexing.py

```
a = np.array([10, 20, 30, 40, 50])

mask = a > 25
print(mask)              # [False False  True  True  True]
print(a[mask])           # [30 40 50] — 조건에 맞는 값만

print(a[[0, 2, 4]])      # [10 30 50] — 원하는 위치만 선택
```

브로드캐스팅이란

개념

같은 크기끼리 연산

- 크기가 같은 배열은 요소끼리(element-wise) 계산됩니다.
- [1,2,3] + [10,20,30] = [11,22,33]

크기가 다를 때

- 조건이 맞으면 작은 배열이 자동으로 크기를 맞춥니다.
- [1,2,3] + 10 = [11,12,13] — 스칼라가 배열 전체에 적용됩니다.
- 반복문 없이도 배열 전체 연산이 가능해집니다.

배열 연산

코드

array_ops.py

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)              # [11 22 33]
print(a * 2)              # [2 4 6] — 브로드캐스팅
print(b / a)              # [10. 10. 10.]
print(a ** 2)             # [1 4 9]
```

자주 쓰는 통계 함수

코드

항목	설명
`np.sum(a)`	모든 요소의 합을 구합니다.
`np.mean(a)`	평균값을 구합니다.
`np.std(a)`	표준편차를 구합니다.
`np.max(a) / np.min(a)`	최댓값 / 최솟값을 구합니다.
`np.argmax(a)`	최댓값이 위치한 인덱스를 반환합니다.

reshape과 transpose

코드

reshape.py

```
a = np.arange(6)              # [0 1 2 3 4 5]

b = a.reshape(2, 3)           # 2행 3열로 변형
print(b)
# [[0 1 2]
#  [3 4 5]]

print(b.T)                    # 전치(transpose) — 3행 2열
```

배열 결합과 분할

코드

combine.py

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.concatenate([a, b]))    # [1 2 3 4 5 6]
print(np.vstack([a, b]))         # 세로로 쌓기
print(np.hstack([a, b]))         # 가로로 이어붙이기

left, right = np.split(np.arange(6), 2)    # 반으로 분할
```

직접 해보기

실습

TASK

1~20 정수로 4x5 배열을 만들고, 짝수만 골라내 1차원 배열로 출력해보세요.

TIP

np.arange(1,21).reshape(4,5)로 배열을 만들고, 불리언 인덱싱(a[a % 2 == 0])으로 짝수를 뽑을 수 있습니다.

난수 생성하기

코드

random_ex.py

```
rng = np.random.default_rng(seed=42)

print(rng.random(3))            # 0~1 사이 난수 3개
print(rng.integers(0, 10, 5))   # 0~9 정수 5개
print(rng.normal(0, 1, (2,2)))  # 정규분포 난수 2x2
```

NOTE seed를 고정하면 실행할 때마다 같은 난수가 나와, 실험 결과를 재현할 수 있습니다.

벡터와 행렬, 왜 중요한가

개념

벡터 (1차원 배열)

- 이미지 한 장, 문장 하나 등 데이터를 숫자로 표현한 것입니다.
- 예: 임베딩 벡터, RGB 픽셀 값

행렬 (2차원 배열)

- 여러 벡터를 모아놓은 것, 또는 이미지 자체를 표현합니다.
- 신경망의 가중치(weight)도 대부분 행렬로 표현됩니다.
- 행렬곱은 AI 모델 연산의 핵심입니다.

행렬곱 연산

코드

linear_algebra.py

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

print(np.dot(a, b))           # 행렬곱
print(a @ b)                  # @ 연산자로도 동일하게 계산

print(np.linalg.inv(a))       # 역행렬
```

조건에 따라 값 다루기

코드

where_clip.py

```
a = np.array([1, -2, 3, -4, 5])

print(np.where(a > 0, a, 0))     # 음수는 0으로: [1 0 3 0 5]
print(np.clip(a, 0, 3))          # 0~3 범위로 잘라내기
print(np.abs(a))                 # 절댓값
```

자주 쓰는 NumPy 함수

요약

항목	설명
`np.array / zeros / ones`	배열을 생성합니다.
`np.reshape / transpose`	배열의 모양을 바꿉니다.
`np.sum / mean / std`	통계값을 계산합니다.
`np.dot / matmul / @`	행렬곱을 계산합니다.
`np.where / clip`	조건에 따라 값을 다룹니다.

직접 해보기

실습

TASK

3x3 크기의 임의의 행렬 두 개를 만들어 행렬곱을 계산하고, 그 결과의 평균과 최댓값을 구해보세요.

TIP

np.random.default_rng().integers(1,10,(3,3))로 3x3 정수 행렬을 손쉽게 만들 수 있습니다.

NumPy가 파이썬 리스트보다 빠른 이유

개념

파이썬 리스트

- 서로 다른 자료형을 섞어 담을 수 있어 유연합니다.
- 반복문으로 요소마다 계산해야 해 느립니다.

NumPy 배열

- 모든 요소가 같은 자료형이라 메모리를 효율적으로 씁니다.
- 반복문 없이 벡터 연산으로 한 번에 계산해 훨씬 빠릅니다.
- 대량의 이미지·수치 데이터를 다룰 때 필수적입니다.

NumPy 핵심 요약

요약

항목	설명
`ndarray`	NumPy의 핵심 자료구조인 다차원 배열
`shape` / `dtype`	배열의 모양과 자료형을 나타내는 속성
브로드캐스팅	크기가 다른 배열끼리도 자동으로 맞춰 계산하는 규칙
인덱싱 / 슬라이싱	배열의 일부를 선택하는 방법
행렬곱 `(@)`	AI 모델 연산의 핵심이 되는 벡터·행렬 계산

PART

4

Pandas

표 형태의 데이터를 불러오고 탐색·가공·집계하는 방법을 Pandas로 익힙니다.

Pandas란?

Pandas는 엑셀 표와 비슷한 형태의 데이터를 다루는 라이브러리로, 데이터를 불러오고 정리·가공·분석하는 데 널리 쓰입니다.

- **DataFrame**

  행과 열로 이루어진 표 형태의 자료구조입니다.

- **Series**

  하나의 열(1차원 데이터)을 다루는 자료구조입니다.

- **NumPy 기반**

  내부적으로 NumPy 배열을 사용해 빠르게 연산합니다.

- **AI 데이터 전처리**

  모델 학습 전 데이터 정리 단계에서 거의 항상 사용됩니다.

설치하고 Series 만들기

확인

series.py

```
# pip install pandas
import pandas as pd

s = pd.Series([90, 85, 95], index=["국어", "영어", "수학"])
print(s)
print(s["영어"])          # 85
```

DataFrame 만들기

코드

dataframe.py

```
data = {
    "이름": ["지우", "민준", "서연"],
    "점수": [90, 85, 95],
    "학년": [1, 2, 1]
}
df = pd.DataFrame(data)
print(df)
```

CSV 읽고 쓰기

코드

csv_io.py

```
df = pd.read_csv("students.csv")      # CSV 파일 불러오기

df.to_csv("output.csv", index=False) # 저장 (인덱스는 제외)

# 엑셀 파일은 read_excel / to_excel 사용
```

데이터 훑어보기

코드

explore.py

```
print(df.head())        # 앞 5행 미리보기
print(df.tail(3))       # 뒤 3행 미리보기
print(df.info())        # 컬럼별 자료형·결측치 정보
print(df.describe())    # 숫자 컬럼의 통계 요약
print(df.shape)         # (행 개수, 열 개수)
```

열 선택하기

코드

select_cols.py

```
print(df["이름"])              # 한 열 선택 -> Series
print(df[["이름", "점수"]])    # 여러 열 선택 -> DataFrame

print(df.columns)              # 전체 컬럼명 목록
```

행 선택하기 (loc · iloc)

코드

select_rows.py

```
print(df.loc[0])              # 인덱스 라벨 0번 행
print(df.iloc[0])             # 위치상 0번째 행

print(df.loc[0:1, "이름"])    # 0~1행의 이름 열
print(df.iloc[0:2, 0:2])      # 위치 기준 슬라이싱
```

조건으로 데이터 걸러내기

코드

filter.py

```
high_scores = df[df["점수"] >= 90]
print(high_scores)

result = df[(df["점수"] >= 85) & (df["학년"] == 1)]  # 여러 조건
print(result)
```

직접 해보기

실습

TASK

students.csv를 불러와 점수가 90점 이상인 학생만 이름과 점수 열만 뽑아 출력해보세요.

TIP

여러 조건을 합칠 때는 and/or 대신 &, | 를 쓰고, 각 조건을 괄호로 감싸야 합니다.

결측치 관련 함수

코드

항목	설명
`.isna() / .notna()`	값이 비어있는지(NaN) 아닌지 True/False로 확인합니다.
`.dropna()`	결측치가 있는 행(또는 열)을 제거합니다.
`.fillna(값)`	결측치를 지정한 값으로 채웁니다.
`.isna().sum()`	컬럼별 결측치 개수를 세어 확인합니다.

결측치 처리하기

코드

missing.py

```
print(df.isna().sum())           # 컬럼별 결측치 개수

df_filled = df.fillna(0)         # 0으로 채우기
df_mean = df.fillna(df.mean(numeric_only=True))  # 평균으로 채우기

df_dropped = df.dropna()         # 결측치 행 제거
```

새로운 열 추가하기

코드

new_column.py

```
df["합격여부"] = df["점수"].apply(lambda x: "합격" if x >= 80 else "불합격")

df["보너스점수"] = df["점수"] + 5       # 열끼리 연산

print(df)
```

정렬과 중복 제거

코드

sort_dedup.py

```
df_sorted = df.sort_values("점수", ascending=False)    # 점수 내림차순

df_unique = df.drop_duplicates(subset="이름")          # 이름 중복 제거

print(df_sorted.head())
```

groupby로 그룹별 집계하기

코드

groupby.py

```
grouped = df.groupby("학년")["점수"].mean()
print(grouped)                    # 학년별 평균 점수

summary = df.groupby("학년").agg(평균점수=("점수","mean"), 인원=("이름","count"))
print(summary)
```

groupby와 함께 쓰는 집계 함수

코드

항목	설명
`.mean() / .sum()`	그룹별 평균 / 합계를 계산합니다.
`.count()`	그룹별 데이터 개수를 셉니다.
`.max() / .min()`	그룹별 최댓값 / 최솟값을 구합니다.
`.agg({...})`	여러 집계 함수를 한 번에 지정해 적용합니다.

직접 해보기

실습

TASK

학년별 평균 점수와 인원수를 groupby로 구하고, 평균 점수가 가장 높은 학년을 출력해보세요.

TIP

grouped.idxmax()를 쓰면 값이 가장 큰 그룹의 인덱스(라벨)를 바로 구할 수 있습니다.

데이터 합치기 (merge · concat)

코드

merge.py

```
scores = pd.DataFrame({"이름": ["지우","민준"], "점수": [90, 85]})
info = pd.DataFrame({"이름": ["지우","민준"], "학년": [1, 2]})

merged = pd.merge(scores, info, on="이름")    # 공통 열 기준 병합

combined = pd.concat([df1, df2])              # 위아래로 이어붙이기
```

간단히 그래프 그리기

코드

plot.py

```
import matplotlib.pyplot as plt

df.groupby("학년")["점수"].mean().plot(kind="bar")
plt.title("학년별 평균 점수")
plt.savefig("scores.png")      # 이미지로 저장
```

NOTE df.plot()은 내부적으로 matplotlib을 사용합니다. 별도 설치가 필요할 수 있습니다 (pip install matplotlib).

언제 NumPy, 언제 Pandas?

개념

NumPy가 좋은 경우

- 순수한 숫자 배열, 행렬 연산이 중심일 때
- 이미지·텐서처럼 같은 자료형의 대량 수치 데이터

Pandas가 좋은 경우

- 열마다 이름과 의미가 있는 표 형태 데이터
- 결측치 처리, 필터링, 그룹 집계가 필요한 경우
- 엑셀·csv처럼 사람이 읽는 실무 데이터

Pandas 핵심 요약

요약

항목	설명
`DataFrame / Series`	표 전체 / 한 열을 다루는 자료구조
`read_csv / to_csv`	CSV 파일을 읽고 쓰는 함수
`loc / iloc`	라벨 기준 / 위치 기준으로 행을 선택
`fillna / dropna`	결측치를 채우거나 제거하는 함수
`groupby`	그룹별로 데이터를 묶어 집계하는 함수

PART

5

이미지처리 이론

디지털 이미지의 구조부터 필터·컨볼루션 개념까지, AI 비전 모델의 기초가 되는 이미지처리 이론을 다룹니다.

디지털 이미지란?

디지털 이미지는 작은 사각형(픽셀)들이 격자로 모여 만들어진 숫자 데이터입니다. 결국 이미지도 NumPy 배열로 다룰 수 있는 숫자 뭉치입니다.

- **픽셀(pixel)**

  이미지를 구성하는 가장 작은 단위로, 색상 값을 하나씩 가집니다.

- **해상도**

  가로×세로 픽셀 수로, 이미지의 크기와 선명도를 나타냅니다.

- **채널**

  색을 표현하는 성분의 개수 (예: RGB는 3채널, 흑백은 1채널).

- **배열로서의 이미지**

  컬러 이미지는 (높이, 너비, 채널) 모양의 3차원 배열입니다.

주요 색상 공간 비교

색상 공간	설명	활용
RGB	빨강·초록·파랑 3채널의 조합으로 색을 표현	일반적인 사진, 화면 표시
Grayscale	밝기 값 하나(1채널)만으로 명암을 표현	연산량을 줄이고 싶은 전처리 단계
HSV	색상(Hue)·채도(Saturation)·명도(Value)로 표현	특정 색상만 추출하는 작업에 유리

이미지를 배열로 불러오기

코드

load_image.py

```
import cv2
import numpy as np

img = cv2.imread("photo.jpg")      # (높이, 너비, 3) 배열로 로드
print(img.shape)                   # 예: (480, 640, 3)
print(img.dtype)                   # uint8 (0~255 정수)
```

NOTE OpenCV는 기본적으로 채널 순서를 BGR로 읽어옵니다. RGB로 바꾸려면 cv2.cvtColor를 사용합니다.

픽셀 값 확인하고 수정하기

코드

pixels.py

```
pixel = img[100, 200]          # (100,200) 위치의 픽셀 값 [B G R]
print(pixel)                   # 예: [ 80 120 200]

img[0:50, 0:50] = [0, 0, 0]  # 왼쪽 위 50x50 영역을 검은색으로
```

크기 조절과 자르기

코드

resize_crop.py

```
resized = cv2.resize(img, (224, 224))     # AI 모델 입력 크기로 조절

cropped = img[50:200, 100:300]            # 배열 슬라이싱으로 자르기

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)   # 흑백 변환
```

밝기와 대비 조정하기

코드

brightness.py

```
brighter = cv2.convertScaleAbs(img, alpha=1.0, beta=40)    # 밝기 +40

contrast = cv2.convertScaleAbs(img, alpha=1.5, beta=0)     # 대비 1.5배
```

NOTE alpha는 대비(곱셈), beta는 밝기(덧셈)를 조절합니다. 값이 255를 넘으면 자동으로 255에서 잘립니다.

컨볼루션이란

개념

커널(kernel)

- 3x3, 5x5처럼 작은 크기의 숫자 격자입니다.
- 이미지 위를 이동하며 주변 픽셀과 곱하고 더합니다.

왜 중요한가

- 커널 값에 따라 블러·샤프닝·엣지검출 등 다양한 효과를 냅니다.
- CNN(합성곱 신경망)의 핵심 연산이 바로 이 컨볼루션입니다.
- 이미지 처리와 딥러닝을 잇는 핵심 개념입니다.

대표적인 필터 종류

코드

항목	설명
블러 (Blur)	주변 픽셀을 평균내어 이미지를 부드럽게 만듭니다.
샤프닝 (Sharpen)	경계를 강조해 이미지를 또렷하게 만듭니다.
엣지 검출 (Edge)	색이 급격히 변하는 경계선만 뽑아냅니다.

블러·샤프닝 적용하기

코드

filters.py

```
blurred = cv2.GaussianBlur(img, (5, 5), 0)      # 블러 처리

kernel = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharpened = cv2.filter2D(img, -1, kernel)       # 샤프닝 커널 적용
```

Canny 엣지 검출

코드

edge_detection.py

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, threshold1=100, threshold2=200)


cv2.imwrite("edges.jpg", edges)
```

NOTE threshold 값이 낮을수록 더 많은 경계선을 검출하지만, 노이즈도 함께 늘어날 수 있습니다.

이진화(Threshold)

코드

threshold.py

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
# 127보다 밝으면 255(흰색), 어두우면 0(검은색)
```

직접 해보기

실습

TASK

이미지를 흑백으로 변환한 뒤 Canny 엣지 검출을 적용하고, threshold 값을 바꿔가며 결과가 어떻게 달라지는지 비교해보세요.

TIP

엣지 검출 전에 GaussianBlur로 노이즈를 살짝 줄여주면 더 깔끔한 경계선을 얻을 수 있습니다.

이미지 정규화란

개념

왜 필요한가

- 픽셀 값은 보통 0~255 범위의 정수입니다.
- AI 모델은 작은 범위의 실수 입력에서 더 안정적으로 학습됩니다.

대표적인 방법

- 0~1 스케일링: 픽셀 값을 255로 나눕니다.
- 표준화: (값 - 평균) / 표준편차로 변환합니다.
- 모델마다 요구하는 정규화 방식이 다를 수 있습니다.

정규화 코드로 적용하기

코드

normalize.py

```
img_float = img.astype(np.float32)

scaled = img_float / 255.0              # 0~1 범위로 스케일링

mean, std = scaled.mean(), scaled.std()
standardized = (scaled - mean) / std    # 평균 0, 표준편차 1로 표준화
```

데이터 증강(Augmentation) 기법

코드

항목	설명
회전 (Rotate)	이미지를 일정 각도 돌려 다양한 각도의 데이터를 만듭니다.
반전 (Flip)	좌우 또는 상하로 이미지를 뒤집습니다.
무작위 자르기 (Crop)	이미지의 일부분을 무작위로 잘라 사용합니다.
색상 변화 (Color Jitter)	밝기·대비·채도를 무작위로 바꿔 다양성을 높입니다.

이미지처리에서 AI 입력까지

개념

전처리 단계

- 이미지 로드 -> 배열 변환
- 크기 조절, 정규화
- 필요시 증강 적용

모델 입력 단계

- 전처리된 배열을 모델에 전달
- 모델은 배열(텐서) 형태로 연산 수행
- 앞서 배운 NumPy 지식이 그대로 이어집니다.

이미지처리 핵심 요약

요약

항목	설명
픽셀 / 채널	이미지를 구성하는 가장 작은 단위와 색상 성분
cv2.imread / resize	이미지를 불러오고 크기를 조절하는 함수
컨볼루션 / 커널	필터 효과와 CNN 연산의 기반이 되는 개념
Canny / threshold	경계선을 검출하고 이미지를 이진화하는 함수
정규화	픽셀 값을 모델이 학습하기 좋은 범위로 조정하는 과정

WHAT'S NEXT

학습 로드맵

01	데이터 시각화 심화	Matplotlib, Seaborn으로 데이터를 더 다양하게 시각화해보세요.
02	scikit-learn으로 머신러닝	배운 NumPy·Pandas를 바탕으로 분류·회귀 모델을 직접 만들어보세요.
03	딥러닝 프레임워크	PyTorch로 클래스 기반 모델을 설계하고 학습시켜보세요.
04	OpenVINO로 배포	이전 가이드에서 다룬 OpenVINO로 학습한 모델을 최적화·배포해보세요.
05	Streamlit으로 서비스화	만든 모델을 Streamlit 앱으로 감싸 웹에서 바로 써볼 수 있게 만들어보세요.

고맙습니다

기초를 다졌다면, 이제 직접 만들어볼 차례입니다.

작은 실습 하나하나가 쌓여 AI 개발 실력이 됩니다.

Q & A